

Atividade de Construção: Sistema de Votação em Java

Objetivo: Criar um sistema de linha de comando para simular uma eleição simples. Ao final, teremos um programa funcional que demonstra os conceitos de **Classes Abstratas** e **Interfaces**, pilares da Programação Orientada a Objetos.

Conceitos que vamos praticar:

- **Abstração:** Criar um "molde" de **Politico** sem precisar definir todos os detalhes.
- **Herança:** Criar classes específicas como **Prefeito** e **Vereador** que herdam características de **Politico**.
- **Polimorfismo:** Tratar diferentes tipos de políticos (Prefeito, Vereador) da mesma forma, como um **Candidato**.
- **Encapsulamento:** Organizar nosso código em classes com responsabilidades bem definidas.

Parte 1: A Estrutura do Projeto

Antes de escrever qualquer código, vamos planejar a estrutura. Uma boa organização nos ajuda a pensar de forma mais clara.

1.1. O Desenho das Classes

- **Candidato (Interface):** Será nosso "contrato". Qualquer um que queira ser candidato na eleição *deve* assinar esse contrato, ou seja, deve ser capaz de informar seu número, receber votos e se apresentar.
- **Partido (Classe):** Uma entidade para representar os partidos políticos.
- **Politico (Classe Abstrata):** Será o nosso "molde". Todo político tem nome e partido. Esta classe vai implementar o contrato **Candidato**, pois todo político no nosso sistema pode ser um candidato.
- **Prefeito, Vereador, etc. (Classes Concretas):** São as especializações de **Politico**. Elas herdam tudo da classe **Politico** e adicionam a informação específica do seu cargo.
- **UrnaEletronica (Classe):** O cérebro do sistema. Ela será responsável por cadastrar candidatos, receber os votos e apurar os resultados.

1.2. Organização dos Arquivos

Vamos criar a seguinte estrutura de pastas para manter nosso projeto organizado:

```
ProcessoEleitoral/  
├── src/  
│   ├── br/  
│   │   ├── com/  
│   │   │   ├── exemplo/  
│   │   │   │   ├── eleicao/  
│   │   │   │   │   ├── entidades/  
│   │   │   │   │   │   ├── Partido.java  
│   │   │   │   │   │   ├── Eleitor.java  
│   │   │   │   │   │   ├── Politico.java  
│   │   │   │   │   │   ├── Vereador.java  
│   │   │   │   │   │   ├── Prefeito.java
```

```
|   └─ Voto.java
|   └─ interfaces/
|       └─ Candidato.java
|   └─ sistema/
|       └─ UrnaEletronica.java
└─ Main.java
```

Parte 2: Definindo o Contrato (**Interface**)

Vamos começar pelo contrato que define o que é ser um candidato.

Crie o arquivo **Candidato.java** no local indicado na estrutura.

src/br/com/exemplo/eleicao/interfaces/Candidato.java

```
package br.com.exemplo.eleicao.interfaces;

/**
 * Interface que define as ações essenciais de um candidato.
 * É como um contrato que obriga qualquer classe que a implemente
 * a ter os métodos abaixo.
 */

public interface Candidato {
    // TODO: Defina a assinatura do método que retorna o número do candidato (int)
    // TODO: Defina a assinatura do método para receber um voto (void)
    // TODO: Defina a assinatura do método que retorna o total de votos (int)
    // TODO: Defina a assinatura do método para exibir os dados do candidato
    (void)
}
```

Parte 3: Criando as Entidades Base (**Partido** e **Politico**)

Agora, vamos criar a classe **Partido** e o molde **Politico**.

3.1. A Entidade **Partido**

Crie o arquivo **Partido.java**.

src/br/com/exemplo/eleicao/entidades/Partido.java

```
package br.com.exemplo.eleicao.entidades;

/**
 * Representa a entidade Partido Político, com nome e sigla.
 */

public class Partido {
    // TODO: Declare os atributos privados: nome (String) e sigla (String)
    // TODO: Crie um construtor que recebe e inicializa o nome e a sigla
    // TODO: Crie os métodos getters para nome e sigla
}
```

```
}
```

3.2. A Classe Abstrata **Politico**

Esta classe é a base para todos os políticos. Note que ela **implements** **Candidato**, ou seja, ela promete cumprir o contrato que definimos.

Crie o arquivo **Politico.java**.

src/br/com/exemplo/eleicao/entidades/Politico.java

```
package br.com.exemplo.eleicao.entidades;

import br.com.exemplo.eleicao.interfaces.Candidato;

/**
 * CLASSE ABSTRATA que serve como molde para todos os políticos.
 * Ela não pode ser instanciada diretamente ("new Politico() dá erro").
 * Ela já implementa o comportamento padrão de um candidato.
 */

// TODO: Declare a classe como "abstract" e faça ela "implements" a interface
// Candidato
public class Politico {

    // TODO: Declare os atributos protegidos:
    // nome (String)
    // partido (Partido)
    // numeroCandidato (int)
    // totalVotos (int)

    // TODO: Crie um construtor que recebe nome, partido e numeroCandidato.
    // Lembre-se de inicializar totalVotos como 0.

    /*
     * == Implementação da Interface Candidato ==
     * Agora, vamos implementar os métodos exigidos pelo contrato "Candidato"
     */

    // TODO: Implemente o método getNumeroCandidato() (apenas retorne o atributo)

    // TODO: Implemente o método receberVoto() (incremente o atributo totalVotos)

    // TODO: Implemente o método getTotalVotos() (apenas retorne o atributo)

    @Override
    public void exibirDados() {
        System.out.println("Nome: " + this.nome);
        System.out.println("Partido: " + this.partido.getNome() + " (" +
this.partido.getSigla() + ")");
        System.out.println("Número: " + this.numeroCandidato);
    }
}
```

```
// A mágica acontece aqui: chamamos um método que AINDA NÃO EXISTE.  
// As classes filhas serão OBRIGADAS a criá-lo.  
System.out.println("Cargo: " + getCargo());  
}  
  
/*  
 * == Método Abstrato ==  
 * Este é o método que as classes filhas (Prefeito, Vereador)  
 * serão forçadas a implementar.  
 */  
  
// TODO: Declare um método abstrato chamado getCargo que retorna uma String.  
  
}
```

Parte 4: Criando os Políticos (Classes Concretas)

Com o molde `Politico` pronto, criar os tipos específicos de políticos fica muito fácil. Eles só precisam herdar de `Politico` e implementar o método `getCargo()`.

4.1. Classe `Prefeito`

Crie o arquivo `Prefeito.java`.

`src/br/com/exemplo/eleicao/entidades/Prefeito.java`

```
package br.com.exemplo.eleicao.entidades;  
  
// TODO: Faça esta classe "extends" (herdar) da classe Politico  
public class Prefeito {  
  
    // TODO: Crie um construtor que recebe nome, partido e numeroCandidato  
    // e repasse esses valores para o construtor da classe pai (super)  
  
    // TODO: Implemente o método abstrato obrigatório "getCargo"  
    // Ele deve apenas retornar a String "Prefeito"  
  
}
```

4.2. Classe `Vereador`

Crie o arquivo `Vereador.java`.

`src/br/com/exemplo/eleicao/entidades/Vereador.java`

```
package br.com.exemplo.eleicao.entidades;  
  
// TODO: Faça esta classe "extends" (herdar) da classe Politico  
public class Vereador {
```

```
// TODO: Crie um construtor que recebe nome, partido e numeroCandidato
// e repasse esses valores para o construtor da classe pai (super)

// TODO: Implemente o método abstrato obrigatório "getCargo"
// Ele deve apenas retornar a String "Vereador"

}
```

Parte 5: Entidades de Apoio (**Eleitor** e **Voto**)

Precisamos de classes simples para representar quem vota e o voto em si.

Crie os arquivos **Eleitor.java** e **Voto.java**.

src/br/com/exemplo/eleicao/entidades/Eleitor.java

```
package br.com.exemplo.eleicao.entidades;

public class Eleitor {
    // TODO: Declare o atributo de eleitor
    // nome (String)

    // TODO: Declare o construtor de Eleitor, que recebe o nome, apenas.

    // TODO: Declare o método que retorna o nome do eleitor.
}
```

src/br/com/exemplo/eleicao/entidades/Voto.java

```
package br.com.exemplo.eleicao.entidades;

public class Voto {
    // TODO: Declare os atributos de Voto
    // eleitor (Eleitor);
    // numeroCandidatoVotado (int);

    // TODO: Declare o construtor de Voto, que recebe Eleitor e
    numeroCandidatoVotado

    // TODO: Crie o método que retorne o número do candidato votado,
}
```

Parte 6: O Cérebro da Operação (**UrnaEletronica**)

Esta é a classe que vai gerenciar a eleição. Ela usará um **ArrayList** para guardar a lista de candidatos cadastrados.

Crie o arquivo `UrnaEletronica.java`.

`src/br/com/ exemplo/eleicao/sistema/UrnaEletronica.java`

```
package br.com.exemplo.eleicao.sistema;

import br.com.exemplo.eleicao.entidades.Voto;
import br.com.exemplo.eleicao.interfaces.Candidato;
import java.util.ArrayList;
import java.util.List;

public class UrnaEletronica {

    // TODO: Declare a lista de candidatos.
    // Pense bem: qual deve ser o tipo da lista? List<Politico>? List<Prefeito>?
    // Dica: Use a Interface para obter o máximo de flexibilidade!
    private List<Candidato> candidatosRegistrados;

    // (A lista de votos pode ser fornecida)
    private List<Voto> votosComputados;

    public UrnaEletronica() {
        // TODO: Inicialize a lista de candidatos (crie o ArrayList)
        this.votosComputados = new ArrayList<>();
    }

    public void registrarCandidato(Candidato novoCandidato) {
        // TODO: Implemente a lógica para registrar um candidato
        // 1. Verifique se o número do "novoCandidato" já existe na lista
        // "candidatosRegistrados"
        // (use um loop "for-each" para percorrer a lista)
        // 2. Se já existir, imprima um erro e use "return;"
        // 3. Se não existir, adicione o "novoCandidato" à lista e imprima
        // sucesso.

    }

    public void receberVoto(Voto voto) {
        int numeroVotado = voto.getNumeroCandidatoVotado();
        Candidato candidatoEncontrado = null;

        // TODO: Implemente a lógica para encontrar o candidato votado
        // 1. Percorra a lista "candidatosRegistrados" (use um loop "for-each")
        // 2. Se encontrar um candidato com o número igual ao "numeroVotado",
        // armazene-o na variável "candidatoEncontrado" e use "break;"

        if (candidatoEncontrado != null) {
            // TODO: Se o candidato foi encontrado, chame o método dele para
            // receber o voto
            // e adicione o "voto" à lista "votosComputados".

            System.out.println("Voto para o número " + numeroVotado + "
confirmado!");
        }
    }
}
```

```
        } else {
            System.out.println("Voto nulo (candidato " + numeroVotado + " não encontrado).");
        }
    }

    // Os métodos exibirResultados() e getCandidatosRegistrados() podem ser
    // fornecidos prontos.
    public void exibirResultados() { /* ... (código pronto) ... */ }
    public List<Candidato> getCandidatosRegistrados() { return
this.candidatosRegistrados; }
}
```

Parte 7: Juntando Tudo e Executando (Main)

Finalmente, a classe **Main** irá orquestrar tudo: criar a urna, os partidos, os candidatos, simular os votos e exibir os resultados.

Crie o arquivo **Main.java**.

src/br/com/exemplo/eleicao/Main.java

```
package br.com.exemplo.eleicao;

import br.com.exemplo.eleicao.entidades.*;
import br.com.exemplo.eleicao.sistema.UrnaEletronica;

public class Main {
    public static void main(String[] args) {
        // 1. Criar a Urna
        // 2. Criar os Partidos
        // 3. Criar os candidatos (Prefeitos e Vereadores)
        // 4. Registrar os candidatos na urna
        // 5. Criar os eleitores
        // 6. Simular a votação
        // 7. Apurar e exibir os resultados
    }
}
```